3 to vertex 4, at the end of the program called '*graph4.sagews*'.

```
print(G.all_paths(3, 4))
```

Execute the program again, and it will print the following output below the image of the graph.

```
[[3, 0, 1, 2, 4], [3, 0, 2, 4], [3, 4]]
```

Though there is an edge between vertices 3 and 4 (a path of length 1), you can also reach 4 from 3 by going through vertices 0, 1, and 2 (a path of length 4), and by going through vertices 0 and 2 (a path of length 3). Hence, the output shows all three distinct paths from 3 to 4. Please find paths between other pairs of vertices as well to gain a better understanding of the situation.

Before we proceed further, let us try to understand what a *cycle* in a graph is. This is a path that starts and ends at the same vertex, and it doesn't visit any vertex more than once, except for the starting and ending vertex, which are the same. I use the symbol *em dash* (—) to denote an edge between a pair of vertices, say 0 and 2, as follows: 0—2. If you observe *Graph_2* shown in Figure 2, 0—2—1—0 is a cycle of length 3, also known as a *triangle* in graph theory. Notice that 2—1—0—2 and 1—0—2—1 are also the same as the cycle 0—2—1—0. Another cycle in *Graph_4* is 3—0—2—4—3, which is a cycle of length 4. Surprisingly, there exists a cycle of length 5 also in *Graph_4*. That cycle is 3—0—1—2—4—3. The edge 0—2 is called a *chord* of this cycle. A cycle without a chord in the original graph is called an *induced cycle*. Now, add the following lines of code at the end of the program called '*graph4.sagews*'.

```
G.longest_cycle(induced=true)
G.longest_cycle(induced=false)
```

The above two lines of code print the number of vertices in the largest induced cycle and the largest cycle in the graph *Graph_4*. Upon execution, the following output is printed on the screen.

```
longest induced cycle: Graph on 4
vertices
longest cycle: Graph on 5 vertices
```

From the output, we can see that the longest induced cycle in *Graph_4* contains 4 vertices, and thus the cycle is of length 4. The induced cycle of length 4 in *Graph_4* is 0—2—4—3—0. Additionally, from the output, we can see that the longest cycle in *Graph_4* contains 5 vertices, and thus the cycle is of length 5. The induced cycle of length 5 in *Graph_4* is 0—3—4—2—1—0.

Now, let us try to elicit more useful information from graphs. First, let us try to understand what a Eulerian cycle is, which is named after the great Swiss mathematician Leonhard Euler. A Eulerian cycle is a cycle in which every edge is visited exactly once. What is the point of finding a Eulerian cycle in a graph? Well, let us assume the graph in consideration represents the road map of a city. If you are tasked with inspecting the quality of all the roads in the city, how will you proceed? Will you travel randomly, or will you have some strategy? Ideally, you would start from your office and inspect all the roads exactly once by travelling through them and coming back to your office at the end. This sort of cycle in a graph is called a Eulerian cycle. Of course, such a route may not be available in the graph (in this case, through the roads of the city). However, SageMath offers a method called *eulerian_circuit( )*, which will return a list of edges forming a Eulerian cycle if one exists. If no Eulerian cycle is found, the method returns *False*. Keep in mind that cycles are often called circuits in graph theory. Now, add the following line of code at the end of the program called '*graph4.sagews*'.

```
G.eulerian_circuit(labels=False)
```

Since there are no labels on the edges of *Graph_4*, the argument *labels* is set to *False*. Upon execution, the output shown on the screen is *False*. Therefore, there is no Eulerian cycle in this graph. Hence, while inspecting the roads, you will have to travel through at least one road twice. Now, let us check whether *Graph_3* has a Eulerian cycle or not. Upon execution of the modified version of the program '*graph3.sagews*', we can see that even *Graph_3* does not contain a Eulerian cycle. Let us modify the program so that a new graph with a Eulerian cycle is created, thus allowing us to observe the output. Modify Line 2 of '*graph3.sagews*' so that the adjacency matrix shown below is used to obtain a new program called '*graph5.sagews*'.

```
adj_matrix = [
    [0, 1, 0, 0, 0, 0, 0, 1],
    [1, 0, 1, 1, 0, 0, 0, 1],
    [0, 1, 0, 1, 0, 0, 0, 0],
    [0, 1, 1, 0, 1, 1, 0, 0],
    [0, 0, 0, 1, 0, 1, 0, 0],
    [0, 0, 0, 1, 1, 0, 1, 1],
    [0, 0, 0, 0, 0, 1, 0, 1],
    [1, 1, 0, 0, 0, 1, 1, 0]
]
```

Upon execution of the program '*graph5.sagews*' we obtain the graph called *Graph_5*, as shown in Figure 3.

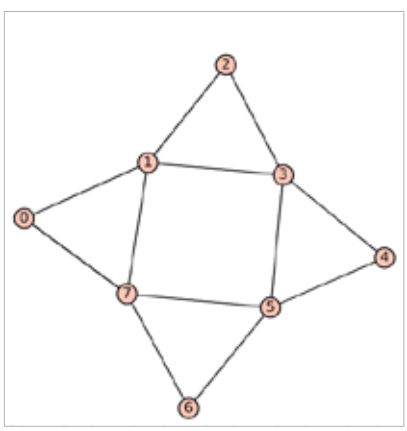

Figure 3: *Graph_5* generated by SageMath